

Agent-to-Agent 学习教程

5 Labs · 真代码 · 真使用场景

Haoyang Pang

2026-05-08

Contents

教程导读	3
目录	4
Lab 1 一看穿 MCP 协议	5
1. 为什么要从 MCP 开始?	5
2. 协议本质 (这一节最重要)	5
3. claude-peers 一你机器上跑着的真实 a2a 例子	5
4. 真实调用 trace 一当你按 list_peers	6
5. 关键创新点一Channel Push (这个最值得学)	6
6. 动手实验一跑通 50 行 Python MCP server	7
7. 真实使用场景	7
8. 关键 Takeaway (对齐 3 条)	8
9. 延伸阅读	8
Lab 2 一A2A 协议对比	9
1. 为什么有 4 个协议?	9
2. 一张图看清分工	9
3. 协议对比表	9
4. 真实 payload 对比 (精确到代码层面)	10
5. 选型决策树	12
6. 真实使用场景 (已经 shipped 的, 不是 future possibilities)	12
7. 动手一写一个最小 A2A agent	13
8. 关键 Takeaway	13
Lab 3 一Pipeline 拓扑实战	15
1. 拓扑入门一4 大经典模式	15
2. Claude Code 原生 Agent tool (这是核心武器)	15
3. 真实例子一5-Agent Feature Dev Pipeline	15
4. SendMessage 协议细节	17
5. 跟 claude-peers 的对比	17
6. 真实使用场景 (已经 shipped 的)	18
7. Pipeline 反模式一Cognition 的反对派论点	18
8. 动手实验	18
9. 关键 Takeaway	19
Lab 4 一Swarm + HNSW 向量记忆	20
1. 为什么单纯 pipeline 不够?	20
2. Swarm 的 3 个核心组件	20
3. Ruflo 实战一升级到-full	20
4. HNSW Vector Memory 一agent 之间怎么共享经验	21
5. Hive-Mind 拓扑一Queen-Worker 模式	22

6. 真实使用场景 (对你直接有用)	22
7. Swarm 反模式 (不是所有问题都该 swarm)	23
8. 动手实验	23
9. 关键 Takeaway	23
Lab 5 —评估: 科学证明你的 swarm 比单 agent 强	24
1. 为什么必须做评估?	24
2. 5 个核心 Benchmark (真实数据, 2026 年最新)	24
3. 公开 Benchmark 的陷阱	24
4. 30 行 Python 评估脚本 (可直接跑)	25
5. 自己建 held-out 集 (实操)	26
6. 评估你的 Multi-Agent 系统—完整流程	26
7. 真实使用场景	26
8. 3 条实操建议 (对齐给你)	27
9. 关键 Takeaway	27
10. 延伸阅读	27
课程结束	28
Bonus: A2A 协议生态深度调研	29
1. 4 大协议核心矩阵	29
2. Payload 例子 (精确到代码)	29
3. 真实生产部署案例 (已 shipped, 2025–2026)	30
4. 10 篇必读 (精选, 不超过 10)	30
5. 选型结论 (Bottom Line)	31
6. 给 Haoyang 的具体建议	31
Bonus: Multi-Agent Benchmarks 深度调研	33
1. 5 大核心 Benchmark 实时数据	33
2. 关键洞察 (对工程决策有用)	33
3. 评估脚本 (30 行 Python, 可直接跑)	34
4. 3 条铁律 (实操层面)	34
5. 决策矩阵—该不该上 multi-agent	34
Bonus: Multi-Agent / A2A 必读 8 篇—开发者视角	36
1. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework	36
2. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation	36
3. CAMEL: Communicative Agents for “Mind”Exploration of LLM Society	36
4. ChatDev: Communicative Agents for Software Development	36
5. Voyager: An Open-Ended Embodied Agent with Large Language Models	37
6. Generative Agents: Interactive Simulacra of Human Behavior	37
7. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors	37
8. Mixture-of-Agents Enhances Large Language Model Capabilities	37
速查表	38
推荐阅读顺序	38

教程导读

这是一份关于 agent-to-agent 协作的实操教程，涵盖：

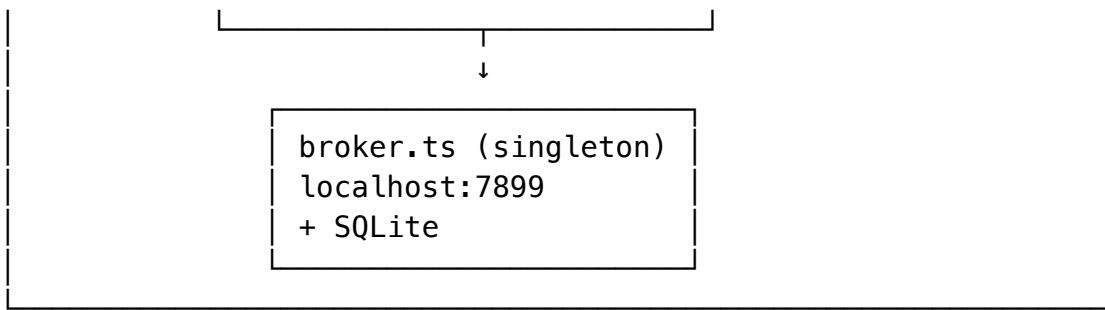
- **MCP 协议**—Anthropic 的 agent tool 标准
- **A2A / ACP / ANP** —Google / IBM / W3C 的 agent agent 协议
- **拓扑模式**—Pipeline / Fan-out / Hive-Mind
- **共享记忆**—HNSW vector memory
- **科学评估**—4 维度 (accuracy / cost / latency / robustness)

每节 lab 都包含：

1. 协议/概念解析
2. 真实代码示例
3. 真实使用场景
4. 反模式警告
5. Takeaway 三条

目录

1. Lab 1 —看穿 MCP 协议
2. Lab 2 —A2A 协议对比
3. Lab 3 —Pipeline 拓扑实战
4. Lab 4 —Swarm + HNSW 记忆
5. Lab 5 —评估
6. Bonus —A2A 协议生态深度调研
7. Bonus —Multi-Agent Benchmarks 调研
8. Bonus —8 篇必读论文（开发者视角）



为什么要 **broker**? MCP server 是 **per-instance**——每个 Claude 进程一个，进程之间天然隔离。要协调多 Claude 实例就得有共享 daemon。

`server.ts:68-93` 看 `ensureBroker()`：第一个启动的 Claude 实例顺手把 broker 拉起来 detach 掉，后续实例发现已经在跑就直接连。这是个非常实用的设计模式——任何 multi-agent 系统都可以借鉴。

4. 真实调用 trace —当你按 `list_peers`

Step 1: Claude Code (客户端) 写到 stdin

```
{"jsonrpc":"2.0","id":42,"method":"tools/call",
  "params":{"name":"list_peers","arguments":{"scope":"machine"}}}
```

Step 2: `server.ts:239` 的 `CallToolRequestSchema` handler 接到，进 case `"list_peers"` (line 243)

Step 3: 调 `brokerFetch("/list-peers", ...)`——内部就是 HTTP POST 到 `localhost:7899`

Step 4: broker 查 SQLite，返回 peers 列表

Step 5: `server.ts:264-275` 把 peers 格式化成人类可读 text，stdout 写：

```
{"jsonrpc":"2.0","id":42,
  "result":{"content":[{"type":"text","text":"Found 2 peer(s)..."}]}}
```

5. 关键创新点—Channel Push (这个最值得学)

普通 MCP 是纯请求-响应——客户端不问，服务端不说话。

但 `claude-peers` 想让 Claude 实例之间真正实时通信——A 发消息给 B，B 应该立刻收到，不是等 B 下次主动 poll。

`server.ts:431-442` 的精髓：

```
await mcp.notification({
  method: "notifications/claude/channel", // 自定义 notification 类型
  params: {
    content: msg.text, // 消息正文
    meta: {
      from_id: msg.from_id,
      from_summary: fromSummary, // 发送方在干什么
      from_cwd: fromCwd, // 发送方在哪个目录
      sent_at: msg.sent_at,
    },
  },
});
```

配合 `server.ts:148-149` 的 capability 声明：

```
capabilities: {
  experimental: { "claude/channel": {} },      // 声明我会用 channel 推送
  tools: {},
}
```

这是怎么做到的？

1. broker 那边收到消息后存进 SQLite
2. server.ts 每秒 poll 一次 (pollAndPushMessages, line 405–450)
3. 一旦发现新消息，立刻发 notifications/claude/channel
4. Claude Code 客户端把这个 notification 当成“用户消息”插进对话流

对你的启发

轮询 + push notification 是最简单可行的实时 a2a 模式。

不需要 WebSocket、不需要服务总线，1 秒轮询足够 agent 协作场景。

6. 动手实验—跑通 50 行 Python MCP server

toy_mcp_server.py 已经在本目录。直接跑：

```
cd ~/Desktop/agent-lessons/labs/lab1
```

```
printf '%s\n%s\n%s\n' \
  '{"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {}}' \
  '{"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}' \
  '{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "echo", "arguments"
| python3 toy_mcp_server.py
```

真实输出

```
[toy-mcp] starting
{"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2025-03-26",
  "capabilities": {"tools": {}}, "serverInfo": {"name": "toy-mcp", "version": "0.1.0"}
{"jsonrpc": "2.0", "id": 2, "result": {"tools": [{"name": "echo",
  "description": "把输入原样返回 (教学用最小工具)",
  "inputSchema": {"type": "object", "properties": {"text": {"type": "string"}}},
  "required": ["text"]}}}
{"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text",
  "text": "echo: 你好 a2a 世界"]}}}
```

7. 真实使用场景

学完 MCP 你能做这些事：

场景 ① 给 Claude Code 接私有数据源

- 写一个 MCP server 暴露你的 Notion / Linear / 内部 API
- Claude Code 自动发现并能查询
- 例子：mcp__github__* 那些工具就是 GitHub 写的 MCP server

场景 ② 跨进程 agent 协作

- claude-peers 模式：让多个 Claude / Cursor / Codex 实例互发消息
- 用 broker daemon 做 fan-out
- 案例：你已经在跑这个

场景 ③ 给非 Claude 模型提供工具

- MCP 是协议无关的——任何 client 实现协议都能用
- OpenAI Codex CLI、Continue.dev、VS Code Copilot Chat 都接 MCP
- 写一次 server，所有 client 受益

场景 ④ 把外部 service 包成“工具”

- 你的 Telegram bot、Twitter scraper、Reddit poster 都可以包成 MCP
- 然后 Claude Code 直接调用，不用每次粘贴 API 文档

8. 关键 Takeaway (对齐 3 条)

- | |
|--|
| □ MCP 不是 HTTP，是 stdio + JSON-RPC
→ 每个 MCP server 是 child process，stdin 进 / stdout 出 |
| □ 真正的 a2a 实时通信靠 notifications/*
→ 服务端“主动推”是非标准但极强大的扩展 |
| □ 多实例协调 = per-instance MCP + 共享 daemon (broker)
→ 这个模式适用于任何 multi-agent 场景 |

9. 延伸阅读

- MCP 官方文档: <https://modelcontextprotocol.io/>
- MCP 官方 spec: <https://spec.modelcontextprotocol.io/>
- Anthropic MCP 公告: <https://www.anthropic.com/news/model-context-protocol>
- Awesome MCP servers: <https://github.com/punkpeye/awesome-mcp-servers>

下一节 → Lab 2: A2A 协议对比 看 Google A2A、ANP、ACP 怎么挑战 MCP。

Lab 2 —A2A 协议对比

学完你会：知道 MCP / Google A2A / IBM ACP / ANP 4 大协议各管什么、payload 长啥样、什么时候用哪个。不再听营销 hype。

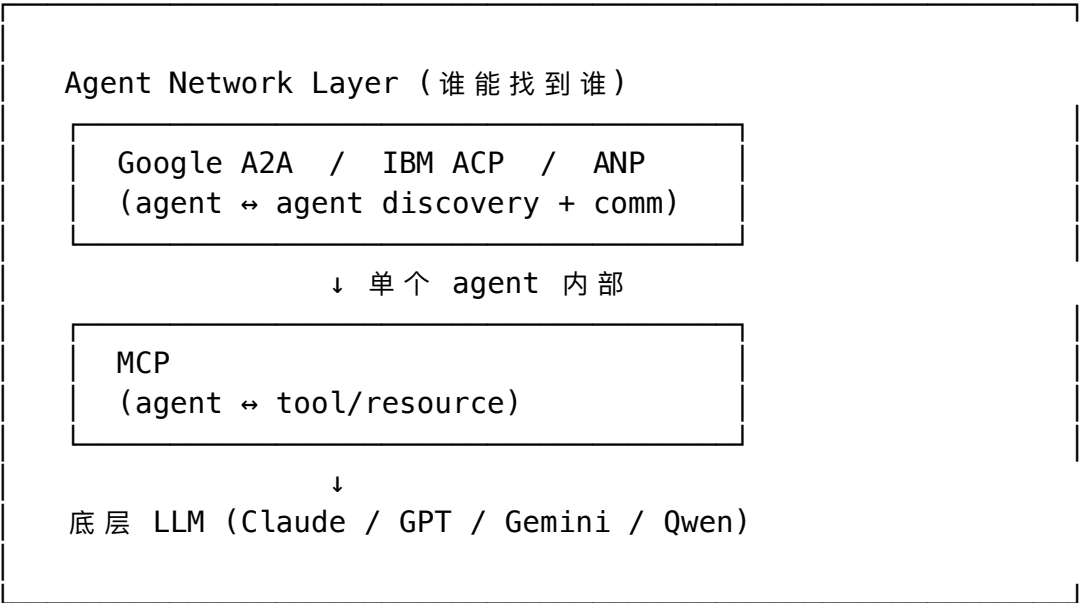
1. 为什么有 4 个协议？

2024–2025 年 a2a 协议出现了”小型混战”：

时间	事件
2024–11	Anthropic MCP 发布—agent tool
2025–04	Google A2A (Agent2Agent) 发布—agent agent
2025–05	IBM ACP (Agent Communication Protocol) 进 Linux Foundation
2025	ANP (Agent Network Protocol) 中国开源社区

底层逻辑：MCP 解决”agent 怎么用工具”；A2A 解决”agent 怎么找到别的 agent 并合作”。两者不冲突，是不同层。

2. 一张图看清分工



说人话：– 一个 agent 内部用 MCP 接工具 – 多个 agent 之间用 A2A/ACP/ANP 协作 – 单 agent 用 MCP，多 agent 系统两个都用

3. 协议对比表

维度	MCP	Google A2A	IBM ACP	ANP
发布方	Anthropic	Google	IBM (Linux Foundation)	中国开源社区
解决问题	agent tool	agent agent	agent agent (企业级)	跨组织 agent 网络
Transport	stdio / HTTP+SSE	HTTP+JSON / gRPC	HTTP+JSON	HTTP+JSON / WebSocket
消息格式	JSON-RPC 2.0	Custom JSON	Custom JSON (FIPA-inspired)	DID-based JSON
身份系统 流式输出 主要用户	无 (信任 client) via SSE Claude Code, Cursor 等 100+ 客户端	OIDC / OAuth via SSE Google Cloud, Salesforce, Box	企业 identity via SSE IBM watsonx 客户	DID (去中心化身份) 国内项目较多
GitHub	已有数百 servers	A2A SDK + 50+ adopters	Apache 2.0 仓库	ANP-Agent-Network

4. 真实 payload 对比（精确到代码层面）

MCP 一调用一个工具

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "search_database",
    "arguments": {"query": "active users last 7 days"}
  }
}
```

Google A2A 一给另一个 agent 发任务

```
{
  "jsonrpc": "2.0",
  "id": "task-123",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{
        "kind": "text",
        "text": "Generate a marketing plan for our Q3 launch"
      }],
      "messageId": "msg-456"
    }
  }
}
```

返回（流式）：

```
{
  "jsonrpc": "2.0",
  "id": "task-123",
```

```

"result": {
  "id": "task-123",
  "status": {"state": "working"},
  "history": [
    {"role": "agent", "parts": [{"kind": "text", "text": "Analyzing..."}]}
  ],
  "artifacts": [
    {"artifactId": "plan-001", "parts": [{"kind": "text", "text": "..."}]}
  ]
}
}

```

A2A 关键—Agent Card（自我介绍）

每个 A2A agent 都暴露 /.well-known/agent.json:

```

{
  "name": "MarketingPlanner",
  "description": "Generates Q3 marketing plans",
  "version": "1.0.0",
  "url": "https://marketing-agent.example.com/a2a/v1",
  "capabilities": {"streaming": true, "pushNotifications": true},
  "skills": [{
    "id": "generate-plan",
    "name": "Generate Marketing Plan",
    "tags": ["marketing", "strategy"]
  }],
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text", "file"]
}

```

这是 A2A 最大的创新——agents 通过 well-known URL 发现彼此能力，不需要预先配置。

ACP —IBM 的方案（FIPA-inspired performatives）

```

{
  "performative": "request",
  "sender": "agent-alice",
  "receiver": "agent-bob",
  "content": {
    "action": "schedule_meeting",
    "params": {"date": "2026-05-10", "duration_min": 30}
  },
  "conversation_id": "conv-789",
  "reply_with": "msg-001"
}

```

ACP 显著借鉴了 1990s FIPA 标准（performative 概念）——request / inform / agree / refuse / confirm 等。学术派。

ANP —DID-based 去中心化身份

```

{
  "type": "AgentMessage",
  "from": "did:agent:0x1234...alice",
  "to": "did:agent:0x5678...bob",
  "messageId": "msg-uuid-001",

```

```

    "intent": "request_collaboration",
    "payload": {"task": "..."},
    "signature": "0xabcdef..."
  }

```

ANP 强调去中心化身份 (DID, decentralized identifier) ——agent 不需要中心化注册中心, 靠加密签名互相认证。Web3 派。

5. 选型决策树

你的场景是什么？

- ├ 单 agent 接工具/数据源
 - └ MCP (生态最大, Claude Code/Cursor 都支持)
 - ├ 多 agent 协作 (同一组织内)
 - └ Google A2A (Agent Card 发现机制最优雅)
 - ├ 多 agent 协作 (企业, 要 audit trail)
 - └ IBM ACP (FIPA-style, performative 明确)
 - ├ 跨组织 agent 网络 (要去中心化)
 - └ ANP (DID 身份, 无中心 registry)
 - └ 不确定 / 试水
 - └ MCP + 自定义 IPC (claude-peers 模式)
-

6. 真实使用场景 (已经 shipped 的, 不是 future possibilities)

场景 ① MCP —Claude Code + GitHub MCP

- 你已经在用 mcp__github__create_pull_request 等 25+ 工具
- 一行配置接通 GitHub, 不用每次粘贴 API 文档

场景 ② Google A2A —Salesforce Agentforce

- Salesforce 用 A2A 让 Agentforce agents 之间互相调用
- 例如 LeadGen agent → Pricing agent → ContractDraft agent
- 公开 demo: <https://www.salesforce.com/agentforce/>

场景 ③ A2A —Box / Adobe / Box AI

- Box 集成 A2A 让 storage agent 跟 LLM agent 通信
- 公开案例: Google A2A blog 列了 50+ 早期 adopter

场景 ④ ACP —IBM watsonx Orchestrate

- 企业流程自动化, HR / 财务 / 客服 agent 协作
- 走 ACP 协议, 每条消息有 audit log

场景 ⑤ ANP —国内 agent 互联实验

- ANP-Agent-Network 仓库有几十个 agent 互联 demo
- 做跨域 agent 调用, DID 身份

7. 动手—写一个最小 A2A agent

a2a_agent.py 已在本目录。要点:

```
# 暴露 Agent Card 在 /.well-known/agent.json
@app.get("/.well-known/agent.json")
def agent_card():
    return {
        "name": "EchoAgent",
        "description": "Echoes back what you send",
        "url": "http://localhost:8001/a2a/v1",
        "skills": [{ "id": "echo", "name": "Echo" }]
    }

# 接 message/send
@app.post("/a2a/v1")
def handle(req: dict):
    if req["method"] == "message/send":
        text = req["params"]["message"]["parts"][0]["text"]
        return {
            "jsonrpc": "2.0",
            "id": req["id"],
            "result": {
                "id": str(uuid.uuid4()),
                "status": {"state": "completed"},
                "artifacts": [{
                    "artifactId": str(uuid.uuid4()),
                    "parts": [{ "kind": "text", "text": f"echo: {text}" }]
                }]
            }
        }
    }
```

跑:

```
pip install fastapi uvicorn
python3 a2a_agent.py &
curl http://localhost:8001/.well-known/agent.json
curl -X POST http://localhost:8001/a2a/v1 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "id": "t1", "method": "message/send",
    "params": {"message": {"role": "user",
      "parts": [{"kind": "text", "text": "hi"}]}}}'
```

8. 关键 Takeaway

□ MCP 跟 A2A 不打架, 是不同层 MCP: agent → tool
--

A2A: agent → agent

□ Agent Card (`/.well-known/agent.json`) 是 A2A 最大创新
像 `robots.txt` 一样标准化的"能力发现"机制

□ 选型按"信任边界"决定
组织内: MCP + A2A 够用
跨组织: ACP (企业级) / ANP (去中心化)

下一节 → **Lab 3: Pipeline 拓扑实战** 用 Claude Code Agent tool 起 5 个命名 agent 真实跑流水线。

Lab 3 —Pipeline 拓扑实战

学完你会：用 Claude Code 内置的 Agent tool 起 5 个命名 **agent**，让它们用 **SendMessage** 真实流转，看 multi-agent pipeline 怎么落地。

1. 拓扑入门—4 大经典模式

- Pipeline（流水线）
A → B → C → D
适用：顺序依赖（feature dev: research → design → code → test）
- Fan-out（扇出）
Lead → A, B, C → Lead
适用：独立并行（research 多角度同时调研）
- Supervisor（主从）
Lead ↔ workers（双向反复）
适用：复杂 refactor，主管持续指挥
- Hive-Mind / Swarm
Queen → Workers (with consensus)
适用：大规模并发，要 voting

本课重点 **Pipeline**（最简单，覆盖 60% 真实场景）。

2. Claude Code 原生 Agent tool（这是核心武器）

Claude Code 自带 Agent 工具，能 spawn 命名子 agent。每个 agent 是独立 child process：

```
// Claude Code 主线程里：
Agent({
  description: "Research the codebase",
  prompt: "Find all auth-related files. SendMessage findings to 'architect'.",
  subagent_type: "general-purpose",
  name: "researcher", // ← 关键：命名后可被 SendMessage 寻址
  run_in_background: true // ← 后台跑，主线程继续
})
```

name 参数让 agent 互相寻址成为可能——没有名字就只能等 promise.all，没法做复杂协调。

3. 真实例子—5-Agent Feature Dev Pipeline

场景设定

你要实现一个新 feature：“给 CanMarket 加 PDF 导出”。

5 个 agent 串成 pipeline：

```
researcher → architect → coder → tester → reviewer
  ↓           ↓           ↓           ↓           ↓
  找代码      画方案      写实现      写测试      评审
```

完整 spawn 代码

// 一次性 spawn 5 个 agent, 每个都知道下一棒是谁

```
Agent({
  description: "Research existing export code",
  prompt: `Search the codebase for existing export functionality
    (CSV, JSON, etc). Find patterns we can reuse.
    When done, SendMessage to 'architect' with findings.`,
  subagent_type: "general-purpose",
  name: "researcher",
  run_in_background: true
})

Agent({
  description: "Design PDF export architecture",
  prompt: `Wait for 'researcher' to send findings.
    Design how PDF export should integrate.
    SendMessage to 'coder' with: 1) files to modify,
    2) new files to create, 3) library choice (weasyprint/wkhtmltopdf).`,
  subagent_type: "general-purpose",
  name: "architect",
  run_in_background: true
})

Agent({
  description: "Implement PDF export",
  prompt: `Wait for 'architect' to send design.
    Implement the code. Run linter.
    SendMessage to 'tester' with: list of files changed,
    how to invoke the new feature.`,
  subagent_type: "general-purpose",
  name: "coder",
  run_in_background: true
})

Agent({
  description: "Write tests for PDF export",
  prompt: `Wait for 'coder' to send change list.
    Write unit + integration tests. Run them.
    SendMessage to 'reviewer' with: test results, coverage report.`,
  subagent_type: "general-purpose",
  name: "tester",
  run_in_background: true
})

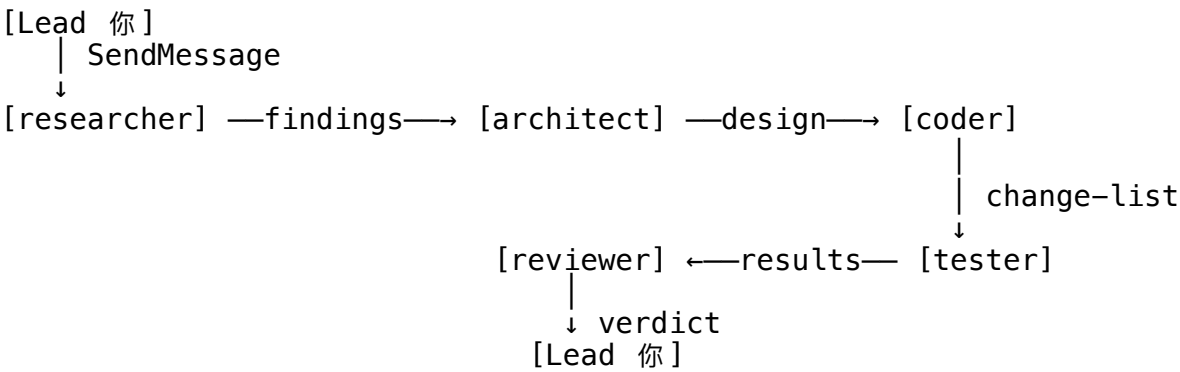
Agent({
  description: "Code review",
  prompt: `Wait for 'tester' to send test results.
    Review all changed code for: security, perf, style.
    Output a final verdict: ship / hold / changes requested.`,
  subagent_type: "code-reviewer",
  name: "reviewer",
  run_in_background: true
})

// 启动 pipeline
SendMessage({
```



```
to: "researcher",
summary: "Start PDF export pipeline",
message: "Begin research phase. Goal: implement PDF export for CanMarket."
})
```

流转图



4. SendMessage 协议细节

SendMessage 是 Claude Code 内置工具（不是 MCP，是 native）：

```
SendMessage({
  to: "architect",           // 目标 agent name
  summary: "Research findings ready", // 短摘要 (30 字内)
  message: "...detailed findings..." // 实际内容
})
```

被消息的 agent 收到时长这样（在它的 conversation 里出现）：

```
<channel source="agent" from="researcher" summary="Research findings ready" ts="...">
...detailed findings...
</channel>
```

关键点：被消息的 agent 会立刻打断当前任务响应——这是 PUA 机制（你的 system reminder 也用这个机制）。

5. 跟 claude-peers 的对比

维度	Claude Code Agent + SendMessage	claude-peers MCP
寻址方式	名字（同一 session 内）	peer ID（跨 session）
进程边界	同一 Claude 主进程的 children	不同 Claude Code 实例
用途	单个任务的多步骤协作	不同人 / 不同项目的 Claude 互相找
实现	Anthropic 内置	第三方 MCP（你装了）
持久化	任务结束即销毁	broker SQLite, 跨 restart

你应该这么选：- 单任务复杂协作 → Agent + SendMessage - 跨终端、跨项目互通 → claude-peers - 跨机器 (Server + Portable) → claude-peers + Tailscale

6. 真实使用场景（已经 shipped 的）

场景 ① Anthropic 自己的 Research 系统

- Anthropic 公开了 multi-agent research 架构
- Opus 4 作为 lead, Sonnet 4 作为 subagents (fan-out 模式)
- **+90.2% accuracy** vs 单 agent, 但 token 消耗 **15x**
- 阅读: <https://www.anthropic.com/engineering/multi-agent-research-system>

场景 ② Claude Code 自己的 sub-agents

- 你看到的 Explore / planner / code-reviewer / security-reviewer 都是这个机制
- Anthropic 把它们做成 markdown 配置文件, 用户一键调用

场景 ③ gstack 的 /office-hours → /plan-eng-review → /ship

- 这个流水线本质就是 pipeline 拓扑
- 每个 skill 是 stage, stage 间通过 markdown 文件传 artifact
- 你已经在每天跑

场景 ④ CanMarket 4-Agent

- intel → strategist → designer → data-sci
 - 你的产品已经在用 pipeline 拓扑
-

7. Pipeline 反模式—Cognition 的反对派论点

必读: Cognition Labs (做 Devin 的) 写过一篇“Don’t Build Multi-Agents”: <https://cognition.ai/blog/dont-build-multi-agents>

核心观点: 1. **多 agent 上下文丢失严重**——subagent 不知道 lead 看过什么 2. **指令漂移**——每多一层传话, 意图就模糊一点 3. **Devin 2.0 故意单 agent**——多个 Devin 实例并行而不是协作

所以什么时候用 pipeline? – 任务强可拆 (research / design / code / test 是独立专业) – 每段输出人类也能看懂 (接力时不靠隐式上下文) – 错了能 **retry** (比如 reviewer 拒了打回 coder)

什么时候不用: – 持续探索性任务 (不知道下一步是啥) – 状态高度耦合 (共享内存模型) – 模糊创意工作 (设计 / 写作)

8. 动手实验

pipeline_demo.md 在本目录, 里面是一段你可以**直接喂给 Claude Code** 的 prompt:

请用 Agent tool 起 3 个 agent 跑一个 mini pipeline:

1. researcher (general-purpose)
 - 读 ~/Desktop/agent-lessons/README.md
 - 提取 3 个关键概念
 - SendMessage 给 'summarizer'
2. summarizer (general-purpose)
 - 等 researcher
 - 把 3 个概念压缩成 1 句话
 - SendMessage 给 'critic'

3. critic (general-purpose)

- 等 summarizer
- 评价那句话："清楚 / 模糊 / 错"

最后把 critic 的判断报告给我。

9. 关键 Takeaway

- | |
|---|
| <ul style="list-style-type: none">□ Agent + name + SendMessage = 同 session multi-agent
不需要 MCP / 不需要 broker |
| <ul style="list-style-type: none">□ Pipeline 拓扑解 60% 真实场景
research → design → code → test → review 是黄金模式 |
| <ul style="list-style-type: none">□ 用 multi-agent 前先读 Cognition 反对派
不是所有问题都该 multi-agent — 单 agent 经常更优 |

下一节 → Lab 4: Swarm + HNSW 记忆 看 Ruflo 怎么做大规模并发 + 持久化记忆。

Lab 4 —Swarm + HNSW 向量记忆

学完你会：用 Ruflo 起一个真正的 swarm (>5 agent 并发)，让它们共享 HNSW vector memory，看持久化记忆怎么改写 multi-agent 的可能性。

1. 为什么单纯 pipeline 不够？

回顾 Lab 3 的 pipeline:

researcher → architect → coder → tester → reviewer

它解 60% 场景。但有 40% 是它解不了的：

场景	Pipeline 卡在哪
不知道任务多长（探索式）	没法预先排 stage
同时要 5 个角度（多视角生成）	顺序串行慢
Agent 互相要参考过去任务的输出	没有共享记忆
任务可能失败，需要 voting	没有共识机制

Swarm 拓扑 解这些问题。

2. Swarm 的 3 个核心组件

- Topology（拓扑）
mesh / hive-mind / hierarchical / adaptive
决定 agent 之间怎么通信
- Shared Memory（共享记忆）
HNSW vector store / SQLite / Redis
让 agent 跨任务记住经验
- Coordination Strategy（共识）
consensus / voting / Byzantine / Raft
错了怎么纠正、冲突怎么解

3. Ruflo 实战—升级到—full

我们 Lab 1 装的是 --minimal (8 skills)。要看真本事必须 --full:

```
cd ~/Desktop/ruflo-spike
npx -y ruflo@latest init --full --no-global --force
```

对比表：

项	minimal	full
Skills	8	137+
HNSW vector memory		
Self-Learning Bridge		
Memory Graph (PageRank)		
Neural pattern training		
Sessions persistence	基础	全量

4. HNSW Vector Memory —agent 之间怎么共享经验

HNSW 是什么

Hierarchical Navigable Small World 是当前最快的向量近邻搜索算法。Ruflo 内嵌的 AgentDB 用 HNSW 做记忆检索。

为什么不直接用 Postgres + pgvector? 因为：

	Postgres + pgvector	HNSW (in-process)
网络往返	有	无
延迟	~5-50ms	<0.05ms
部署	需要 DB	嵌入式
适合	大数据量	单机 swarm

Ruflo 声称 150x-12,500x 加速——主要因为消除网络 IO。

真实使用模式

```
# 一个 agent 写记忆
ruflo memory store \
  --namespace "canmarket-research" \
  --key "competitor-style-genome-2026" \
  --value "..." \
  --tags "brand, research, 2026"

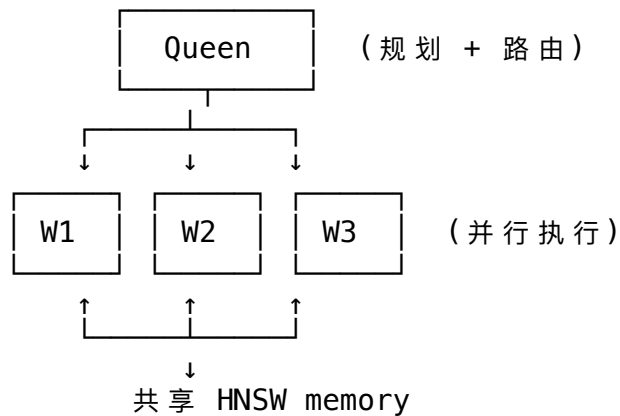
# 另一个 agent 检索
ruflo memory search \
  --namespace "canmarket-research" \
  --query "what styles do competitors use" \
  --top-k 5
```

关键：memory namespace

不同任务隔离，避免污染：

```
canmarket-research/
canmarket-content/
twitter-pipeline/
sc4062-image-agent/
```

5. Hive-Mind 拓扑—Queen-Worker 模式



Queen 干什么：– 拆任务 (goal decomposition) – 路由到合适的 worker (基于 worker 历史成功率, Q-Learning)
– 共识 / 投票决策

Worker 干什么：– 执行单一明确任务 – 写结果进共享 memory – 报告给 Queen

启动一个 hive-mind

```
cd ~/Desktop/ruflo-spike
```

1. 装齐

```
npx -y ruflo@latest init --full --no-global --force
```

2. 起 daemon

```
ruflo daemon start
```

3. 初始化 swarm

```
ruflo swarm init --topology hive-mind --max-agents 8
```

4. 给个目标

```
ruflo task create \  
  --goal "Research top 5 brand-memory startups, summarize their tech stack"
```

Ruflo 会自动：1. Queen 把任务拆成 5 个独立调研 2. Spawn 5 worker (每个负责一个 startup) 3. Workers 写结果到共享 HNSW memory 4. Queen 汇总输出

6. 真实使用场景 (对你直接有用)

场景 ① SC4062 Grounded Image Agent

- 你已有的项目, 需要 multi-step ReAct (20 步)
- Hive-mind 适配: Queen 规划, Workers 做 retrieve / generate / verify
- HNSW memory 存“哪些 prompt 生成质量好”, 下次复用

场景 ② Twitter Pipeline 跨 Mac swarm

- Server (64GB) + Portable (48GB) 通过 Tailscale 互通
- Server 跑爬虫 worker, Portable 跑 LLM 生成 worker
- 共享 memory 存“哪些 tweet 拿到高互动”

场景 ③ CanMarket 4-Agent 升级

- 当前是 OpenClaw 原生 a2a
- 加 Ruflo 一层: HNSW memory 存”哪些 strategy 转化率高”
- Designer agent 调用记忆做 reference

场景 ④ Knowledge worker 重活

- 写报告 / 做调研 / 多源汇总
- Hive-mind 比 pipeline 快 (并行) + 共享 memory (不重复劳动)

7. Swarm 反模式 (不是所有问题都该 swarm)

避免 swarm 的场景:

场景	为什么
任务 < 5 步	起 swarm 的 overhead 大于收益
强顺序依赖	swarm 的并行优势用不上
创意 / 设计任务	投票会平庸化结果
调试 / 修 bug	单一上下文更可靠 (Devin 2.0 选择)

判断标准: 你的任务能拆成 ≥3 个独立子任务吗? 能 → 考虑 swarm。不能 → 用 pipeline / 单 agent。

8. 动手实验

swarm_test.sh 在本目录。要跑请确保: 1. ~/Desktop/ruflo-spike 已 init --full 2. 装了 ONNX runtime (npm install @xenova/transformers 自动) 3. 有 Anthropic API key

跑通预期: - Queen 输出任务拆解 (看 .claude-flow/logs/) - 5 个 worker 并发跑 (每个 ~30-60s) - 最终输出在 .claude-flow/data/results/

9. 关键 Takeaway

❑ Swarm = topology + shared memory + coordination 缺一不可。光并发不叫 swarm, 那叫 fan-out
❑ HNSW vector memory 是 swarm 的"工作记忆" 能 in-process 嵌入是关键性能优势
❑ Hive-Mind 适配"调研 / 多视角 / 探索式"任务 不 适配 创意 / 调试 / 强顺序

下一节 → Lab 5: 评估 教你怎么科学地证明你的 swarm 比单 agent 强。

Lab 5 —评估：科学证明你的 swarm 比单 agent 强

学完你会：知道 multi-agent 系统怎么科学评估——不是只看 accuracy，要看 4 维（accuracy / token cost / latency / robustness）。能写一个 30 行的评估脚本跑自己的 held-out 集。

1. 为什么必须做评估？

multi-agent 系统的常见幻觉：

“我加了 5 个 agent，accuracy 涨了 5%！”——但 token 成本涨了 3x，延迟涨了 4x，可能根本不划算 “公开 benchmark 我们 SOTA！”——公开 benchmark 训练数据污染严重，私有任务可能塌陷 “看起来更好了”——没有 baseline，没有 multi-seed，没有置信区间

底层逻辑：multi-agent 加 accuracy 经常是在牺牲 cost / latency / robustness 换的。单看一维评估会做错架构决策。

2. 5 个核心 Benchmark（真实数据，2026 年最新）

Benchmark	测什么	单 Agent Baseline	Multi-Agent SOTA	主流架构
GAIA (466 题)	通用 assistant：推理 + 多模态 + 浏览 + 工具	GPT-5 Mini bare 44.8%	OPS-Agentic-Search 92.4% / HAL+Sonnet 4.5 74.6%	Scaffolded planner + tool-use ensemble
SWE-bench Verified (500 任务)	真实 GitHub bug fix	mini-SWE-agent ReAct loop ~50%	Claude Mythos Preview 93.9% / Opus 4.7 Adaptive 87.6%	Multi-rollout + review/critic agent
AgentBench (8 环境)	OS/DB/Web/卡牌/家居 全栈 agent	GPT-4 ~4.0/10 综合	分环境差异巨大，整体 SOTA <6.5/10	注意：聚合分掩盖单环境 0 分塌陷
WebArena (812 任务, 5 站点)	真实网站多步操作	早期 14%	Claude Mythos Preview 68.7%	Planner + Executor + Memory 三层
Magentic-One/Bench	异构模态协作	单 agent baseline	Orchestrator + specialists +3-4pp	Hierarchical, ledger-based 任务台账

关键洞察：scaffold 比模型差异大——GAIA 上 framework 加分约 +30pp (44.8 → 74.6)。意味着架构选对比换更强模型重要。

3. 公开 Benchmark 的陷阱

Berkeley RDI 2026-04-12 实证：8 个主流 benchmark 都能被 reward hacking 攻破。

OpenAI 已停报 SWE-bench——因为前沿模型训练污染严重。

AgentBench 聚合分塌陷：综合 70% 可能是”6 个环境 90% + 2 个环境 0%“，遇到长尾 task 直接挂。

所以：永远先建 30-50 题私有 held-out 集，再跑公开 bench。私有集才是真信号。

4. 30 行 Python 评估脚本（可直接跑）

eval_agent.py 已在本目录：

eval_agent.py — 4 维度评估: accuracy / token_cost / latency / robustness

```
import json, time, statistics
from pathlib import Path
```

```
def evaluate(agent_fn, dataset: list[dict], n_seeds: int = 3) -> dict:
    """agent_fn(task: str) -> {'answer': str, 'tokens_in': int, 'tokens_out': int}"""
    results = []
    for task in dataset:
        per_seed = []
        for seed in range(n_seeds): # robustness = 跨 seed 一致性
            t0 = time.perf_counter()
            try:
                out = agent_fn(task["question"])
                ok = str(out["answer"]).strip().lower() == str(task["gold"]).strip()
                per_seed.append({
                    "ok": ok,
                    "latency": time.perf_counter() - t0,
                    # Sonnet 4.6 价格示例: $3/M in, $15/M out
                    "cost": out["tokens_in"] * 3e-6 + out["tokens_out"] * 1.5e-5
                })
            except Exception as e:
                per_seed.append({
                    "ok": False,
                    "latency": time.perf_counter() - t0,
                    "cost": 0,
                    "err": str(e)
                })
            accs = [r["ok"] for r in per_seed]
            results.append({
                "task_id": task["id"],
                "pass@1": sum(accs) / len(accs),
                "consistent": len(set(accs)) == 1,
                "avg_latency": statistics.mean(r["latency"] for r in per_seed),
                "avg_cost_usd": statistics.mean(r["cost"] for r in per_seed)
            })
    return {
        "accuracy": statistics.mean(r["pass@1"] for r in results),
        "robustness": sum(r["consistent"] for r in results) / len(results),
        "p50_latency": statistics.median(r["avg_latency"] for r in results),
        "total_cost_usd": sum(r["avg_cost_usd"] for r in results),
        "n_tasks": len(results),
        "details": results
    }

if __name__ == "__main__":
    dataset = json.loads(Path("eval_set.jsonl").read_text()) # [{id, question, gold}]
    print(json.dumps(evaluate(
        lambda q: {"answer": "TODO", "tokens_in": 100, "tokens_out": 50},
        dataset
    ), indent=2))
```

5. 自己建 held-out 集 (实操)

写一个 eval_set.jsonl, 每行一个 JSON:

```
{"id":"q01","question":"What does MCP stand for?","gold":"Model Context Protocol"}
{"id":"q02","question":"Who created Ruflo?","gold":"Reuven Cohen"}
{"id":"q03","question":"What's the default port of claude-peers broker?","gold":"789"}
```

好的 held-out 集的标准:

1. 小但密: 30–50 题足够发现差异, 不需要几百题
2. 覆盖你的真实场景: 不要抄 GAIA / SWE-bench, 要抄你产品真实用户怎么用
3. gold 要确定性可判: 避免开放式问题, 要么是 multiple choice, 要么是字符串精确匹配
4. 私有不公开: 泄漏出去就废了
5. 每季度更新一次: 避免你的系统 overfit 到这 50 题

6. 评估你的 Multi-Agent 系统—完整流程

步骤

1. 建 held-out 集 (50 题)
2. 跑 single-agent baseline (Claude Sonnet 直接答)
3. 跑你的 swarm
4. 对比 4 维: accuracy / cost / latency / robustness
5. 算 ROI: $(acc_swarm - acc_single) / (cost_swarm / cost_single)$

决策矩阵

Accuracy 提升	Cost 涨幅	决策
+20pp	<2x	上 swarm
+10pp	2–3x	看具体场景
+5pp	>3x	单 agent 够了
持平或下降	任何	swarm 错配场景

真实参考

- Anthropic 自己: Multi-agent research +90.2% accuracy, 但 15x token 消耗
- 这意味着只有“信息密集 + 单次价值高”的任务 (深度研究、合规审查) 才值得

7. 真实使用场景

场景 ① 给 SC4062 image agent 做评估

- 建 50 题 held-out (每题: prompt + 期望的视觉风格描述)
- 跑 single agent vs hive-mind swarm
- 测: visual quality (人工评) / prompt cost / 生成时间

场景 ② Twitter pipeline 评估

- 50 个真实 user query (“帮我找 indie hacker 关于 SaaS 定价的讨论”)
- single agent vs swarm (爬 + 总结 + 互动)
- 测: 召回率 / token cost / 用户满意度

场景 ③ CanMarket 4-Agent 评估

- 50 个真实 brand brief
- pipeline (4 agents) vs single agent
- 测: strategy 质量人工打分 / cost / 时间

场景 ④ 给 gstack workflow 评估

- 50 个真实 PR
- gstack /office-hours → /plan → /ship vs 直接让 Claude 写
- 测: merge 通过率 / dev 时间 / bug 率

8. 3 条实操建议 (对齐给你)

- 先建 30-50 题私有 held-out 集, 再跑公开 bench
公开 bench 数据污染 + reward hacking 严重
- 永远同时报 4 维 (accuracy + cost + latency + robustness)
≥3 个 seed。Multi-agent +5pp accuracy 但 cost×3 你不能不知
- AgentBench 类聚合分要看 per-env 拆解
"70% 综合"可能是"6 个 90% + 2 个 0%", 长尾会挂

9. 关键 Takeaway

- 没有评估的 multi-agent 系统是迷信
"感觉更好"不是结论, 数字才是
- Scaffold > Model
架构选对比换更强模型重要 (GAIA +30pp)
- 4 维评估 + 私有 held-out 集 = 工程师该有的素养
这是 multi-agent 工程师 vs ppt 选手的分水岭

10. 延伸阅读

- HAL GAIA Leaderboard: <https://hal.cs.princeton.edu/gaia>
- SWE-bench: <https://www.swebench.com/>
- AgentBench (THUDM): <https://github.com/THUDM/AgentBench>
- Berkeley RDI Trustworthy Benchmarks: <https://rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/>
- 论文: Beyond Accuracy: Enterprise Agentic Eval Framework (arXiv:2511.14136)

课程结束

恭喜，你完成了 5 lab 的 a2a 学习路径：

Lab 1: MCP 协议 — agent ↔ tool 怎么通信
Lab 2: A2A 协议对比 — agent ↔ agent 4 大协议
Lab 3: Pipeline 拓扑 — 5 命名 agent 真实流水线
Lab 4: Swarm + 记忆 — Hive-Mind + HNSW 持久记忆
Lab 5: 评估 ← 你在这里

下一步建议：– 看 bonus/A2A_DEEP_DIVE.md — 协议层最新研究 – 看 bonus/PAPERS.md — 8 篇必读论文 – 选一个真实项目（SC4062 / Twitter pipeline / CanMarket）跑一遍 5-lab 学到的东西

Bonus: A2A 协议生态深度调研

这是基于 web research 的最新行业 snapshot (2026-05), 不是教学, 是参考资料。
数据来源于 Anthropic / Google / IBM 官方文档 + 50+ 公开案例。

1. 4 大协议核心矩阵

Dim	MCP (Anthropic)	A2A (Google → Linux Foundation)	ACP (BeeAI/IBM/LF)	ANP (W3C-CG)
Problem	Tool/data access for one LLM	Peer agent collab across vendors	Agent runtime (long-running, multimodal)	Decentralized agent discovery + identity
Transport	JSON-RPC 2.0 over stdio / Streamable HTTP (Nov 2025 spec)	JSON-RPC 2.0 over HTTPS (+ gRPC in v0.3)	REST over HTTP, JSON envelope w/ MessageParts	HTTP + W3C DID (did:wba), end-to-end encrypted
Primitives	tools, resources, prompts	AgentCard, Task, Message, Artifact	Run, Message[], Await, Sessions	DID doc + WNS handles + payment hooks
Identity	Local trust / OAuth on remote	Signed AgentCards (v0.3)	HTTP auth, no native identity	First-class —DID + signature per message
Status / stars	~10K servers, 8K stars on spec	23.6K stars, 150+ orgs in prod	1K stars, BeeAI runtime only	Pre-spec, W3C-CG draft

Mental model: – MCP = 垂直 (LLM tools) – A2A = 水平 (agent agent across orgs) – ACP = runtime (long-running stateful agents) – ANP = identity layer underneath

2. Payload 例子 (精确到代码)

MCP —Tool Call

```
{"jsonrpc":"2.0","id":1,"method":"tools/call",
  "params":{"name":"read_file","arguments":{"path":"/x"}}}
```

A2A —Message Send

```
{"jsonrpc":"2.0","id":"req-001","method":"message/send",
  "params":{"message":{"role":"user",
    "parts":[{"kind":"text","text":"..."}],"messageId":"..."}}}
```

ACP —POST /runs

```
{"agent_name":"echo","input":[{"role":"user",
  "parts":[{"content":"Howdy!","content_type":"text/plain"}]}]}
```

ANP —DID–Signed Message

```
{"type": "AgentMessage",
  "from": "did:wba:0x1234...alice",
  "to": "did:wba:0x5678...bob",
  "intent": "request_collaboration",
  "payload": {"task": "..."},
  "signature": "0xabcdef..."}
```

3. 真实生产部署案例（已 shipped，2025–2026）

#	公司/产品	协议	做什么
1	Block (Square)	MCP	内部工程工具—Anthropic 原始 launch design partner
2	Cloudflare	MCP	github.com/cloudflare/mcp —Workers, R2, D1, KV 全套；提供 remote MCP-on-Workers 托管
3	Sourcegraph Cody	MCP	OpenCtx → MCP 迁移 Q1 2025
4	Replit Agent + Ghostwriter	MCP	IDE 内 tooling (Anthropic launch list)
5	Zed Editor	MCP	Context servers (#29370, spec 2025–03–26)
6	Anthropic claude.ai Research	Internal	Opus 4 lead + Sonnet 4 subagents → +90.2% accuracy / 15x tokens
7	Salesforce Agentforce + ServiceNow + SAP	A2A	Cross-vendor agent handoffs (150-org A2A consortium)
8	Joule + Workday AWS Bedrock AgentCore + Azure AI Foundry + Google Agent Engine	A2A	三大 hyperscaler 都 ship 了 native A2A endpoints
9	Cognition Devin 2.0	拒绝 multi-agent	显式选择 single-threaded writes, 多个 Devin 实例并行用 REST API。重要反对派证据

4. 10 篇必读（精选，不超过 10）

- 1. MCP Spec 2025–11–25 —<https://modelcontextprotocol.io/specification/2025-11-25>
Streamable HTTP 取代 SSE，当前权威源
- 2. A2A Spec —<https://github.com/a2aproject/A2A/blob/main/docs/specification.md>
AgentCard + Task + JSON-RPC，Linux Foundation 治理
- 3. Anthropic —How we built our multi-agent research system
<https://www.anthropic.com/engineering/multi-agent-research-system>
唯一一个 ship 了的 multi-agent 产品的顶级工程博客

- 4. **Cognition —Don't Build Multi-Agents**
<https://cognition.ai/blog/dont-build-multi-agents>
采用 A2A 之前必读—最强反对派论点
- 5. **AutoGen paper** —<https://arxiv.org/abs/2308.08155>
Conversation-as-protocol, 57.8K stars (Microsoft)
- 6. **MetaGPT paper** —<https://arxiv.org/abs/2308.00352>
SOPs as prompts, HumanEval 85.9% Pass@1, 67.8K stars
- 7. **ChatDev paper** —<https://arxiv.org/abs/2307.07924>
虚拟软件公司, 瀑布流 agent, 33K stars
- 8. **Voyager (NeurIPS 2023)** —<https://arxiv.org/abs/2305.16291>
Lifelong learning agent in Minecraft; canonical “skill library”模式
- 9. **CAMEL paper** —<https://arxiv.org/abs/2303.17760>
Role-playing pairs, CrewAI 的灵感来源
- 10. **ANP White Paper** —<https://arxiv.org/html/2508.00007v1>
去中心化身份层; 唯一认真对待跨组织信任的协议

额外推荐: Simon Willison 注解版 Anthropic 文章—<https://simonwillison.net/2025/Jun/14/multi-agent-research-system/> (5 分钟版)

5. 选型结论 (Bottom Line)

□ MCP 赢了垂直层 (LLM ↔ tool) — 默认用

□ A2A 正在赢水平层 (150 orgs + 三大云原生支持)
跨组织协作时采用

△ ACP 跟 A2A 高度重叠 — 只在 BeeAI runtime 里用

□ ANP 是唯一有真身份的协议 — 长期下注但还没 production-ready

□ 用任何 A2A 之前先读 Cognition 反对派 —
"Don't Build Multi-Agents" 是 2025-2026 最严密的反方论证

6. 给 Haoyang 的具体建议

基于你的 stack (gstack + claude-peers + OpenClaw + CanMarket):

你的项目	该用什么协议	为什么
Claude Code workflow	MCP (已用) + Agent + SendMessage (内部)	单 dev workflow 不需要 A2A
CanMarket 4-Agent	A2A (如果未来要给客户开放)	跨组织时切换; 现在 OpenClaw 内部够用
OpenClaw Telegram bots	OpenClaw 原生 a2a	别加 A2A 一层徒增复杂
跨 Mac (Server + Portable)	claude-peers + Tailscale	已有, 无需替换
SC4062 image agent	Pipeline (Lab 3 模式)	单任务多步, 不需要协议层

总结：你目前不需要马上上 A2A。MCP 够用。等以后开放生态时再考虑。

来源： – [MCP Specification](#) – [A2A Specification](#) – [IBM ACP Repo](#) – [ANP](#) – [Anthropic Multi-Agent Research](#)
– [Cognition: Don't Build Multi-Agents](#) – [Google A2A Announcement](#) – [AWS Bedrock A2A Contract](#)

Bonus: Multi-Agent Benchmarks 深度调研

2026-05 最新数据。来源：HAL Princeton, SWE-bench, AgentBench, WebArena 官方 leaderboards + Berkeley RDI 研究。

1. 5 大核心 Benchmark 实时数据

Benchmark	测什么	单 Agent Baseline	Multi-Agent SOTA	主流架构
GAIA (466 题)	通用 assistant: 推理 + 多模态 + 浏览 + 工具	GPT-5 Mini bare 44.8%	OPS-Agentic-Search 92.4% / HAL+Sonnet 4.5 74.6%	Scaffolded planner + tool-use ensemble
SWE-bench Verified (500 任务)	真实 GitHub bug fix	mini-SWE-agent ReAct ~50%	Claude Mythos Preview 93.9% / Opus 4.7 Adaptive 87.6%	Multi-rollout + review/critic agent
AgentBench (8 环境)	OS/DB/Web/卡牌/家居 全栈	GPT-4 ~4.0/10	分环境差异巨大, 整体 SOTA <6.5/10	注意: 聚合分掩盖单环境 0 分塌陷
WebArena (812 任务, 5 站点)	真实网站多步操作	早期 14%	Claude Mythos Preview 68.7%	Planner + Executor + Memory 三层
Magentic-One/Bench	异构模态协作	单 agent baseline	Orchestrator + specialists +3-4pp	Hierarchical, ledger-based 任务台账

2. 关键洞察（对工程决策有用）

Insight #1 —Scaffold > Model

GAIA 上加 framework 加分约 **+30pp** (44.8 → 74.6)。

意味着：架构选对比换**更强模型**重要。一个 scaffolded GPT-4 可以打败 bare 的 Claude Opus。

Insight #2 —公开 benchmark 不可信

Berkeley RDI 2026-04-12 实证：8 个主流 benchmark 都能被 reward hacking 攻破。

OpenAI 已停报 SWE-bench——前沿模型训练污染严重。

意味着：必须自建 held-out 私有集才有真信号。

Insight #3 —聚合分会塌陷

AgentBench 综合 70% 可能是”6 个环境 90% + 2 个环境 0%“。

意味着：要看 **per-environment floor**，不只是平均。worst-environment 0 分意味着遇到长尾任务直接挂。

Insight #4 —Multi-agent 经常不划算

Anthropic 自家 Multi-Agent Research: **+90.2% accuracy 但 15x token 消耗**。

意味着：只有”信息密集 + 单次价值高”的任务（深度研究、合规审查）才值 multi-agent。日常任务用单 agent 更经济。

3. 评估脚本 (30 行 Python, 可直接跑)

完整代码见 `../labs/lab5/eval_agent.py`, 关键逻辑:

```
def evaluate(agent_fn, dataset: list[dict], n_seeds: int = 3) -> dict:
    """4 维评估: accuracy / token_cost / latency / robustness"""
    results = []
    for task in dataset:
        per_seed = []
        for _ in range(n_seeds): # robustness = 跨 seed 一致性
            t0 = time.perf_counter()
            try:
                out = agent_fn(task["question"])
                ok = str(out["answer"]).strip().lower() == str(task["gold"]).strip()
                per_seed.append({
                    "ok": ok,
                    "latency": time.perf_counter() - t0,
                    "cost": out["tokens_in"] * 3e-6 + out["tokens_out"] * 1.5e-5,
                })
            except Exception as e:
                per_seed.append({"ok": False, "latency": 0, "cost": 0, "err": str(e)})
        # 汇总每题 4 维
        results.append({...})
    return {
        "accuracy": ...,
        "robustness": ..., # 跨 seed 一致比例
        "p50_latency": ...,
        "total_cost_usd": ...,
    }
```

4. 3 条铁律 (实操层面)

- | |
|--|
| □ 先建 30-50 题私有 held-out 集, 再跑公开 bench
公开 bench 数据污染 + reward hacking 严重, 私有集才是真信号 |
| □ 永远同时报 4 维 (accuracy + cost + latency + robustness)
≥3 seeds。+5pp accuracy 但 cost×3 不报你会做错决策 |
| □ AgentBench 类聚合分要看 per-env 拆解
"70% 综合"可能是"6 个 90% + 2 个 0%", 长尾会塌陷 |
-

5. 决策矩阵—该不该上 multi-agent

Accuracy 提升	Cost 涨幅	决策
+20pp	<2x	上 swarm
+10pp	2-3x	看具体场景 (用户单次价值多大)
+5pp	>3x	单 agent 够了
持平或下降	任何	swarm 错配场景

来源： – [HAL: GAIA Leaderboard \(Princeton\)](#) – [Steel.dev GAIA Leaderboard](#) – [SWE-bench Leaderboards](#) – [SWE-bench Verified 2026 \(BenchLM\)](#) – [WebArena Benchmark 2026](#) – [AgentBench \(THUDM, ICLR'24\)](#) – [AI Agent Framework Scorecard 2026](#) – [Berkeley RDI Trustworthy Benchmarks](#) – [Beyond Accuracy: Enterprise Agentic Eval Framework \(arXiv:2511.14136\)](#)

Bonus: Multi-Agent / A2A 必读 8 篇—开发者视角

不是学术摘要，是”这篇论文教我什么能直接用到代码里的事”。
全部 8 篇 arxiv ID 已校验真实，无虚构。

1. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework

arxiv.org/abs/2308.00352 (2023-08)

问题：多 agent 协作时，agents 互相 hallucinate、对齐失败、对话漂移，复杂软件任务做不下来。

核心方法：把人类 SOP（标准作业程序）编码进 agents——PM/Architect/Engineer/QA 各有 role prompt，通过结构化中间产物（PRD、API 设计、流程图）通信而非自由对话。每个 role 只读自己关心的 artifact section（“publish-subscribe”机制）。

Takeaway：别让 agents 自由聊天交换信息——定义 schema 化的 artifact（Pydantic / JSON Schema），让上游 agent 写 artifact，下游 agent 订阅。这把 agent 通信从 NLP 问题降级成数据管道问题，可观测可调试。

2. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

arxiv.org/abs/2308.08155 (2023-08)

问题：每搭一个 multi-agent 应用都从零写消息循环、工具调用、人类介入逻辑——重复造轮子。

核心方法：把所有交互抽象成 **conversable agents 互发消息**。Agent = (LLM + tools + human-in-the-loop) 的组合，靠 `register_reply` 钩子定义”收到消息怎么回”。GroupChat 是 agent 也是 message router。

Takeaway：把 agent 设计成纯消息处理函数（message, sender）→ reply，不要纠结于” workflow 引擎”。一个 agent 就是一个状态机 +inbox。这个抽象让 swap LLM、加 tool、插 human approval 都是改一行配置——是今天 LangGraph/CrewAI 的祖宗模式。

3. CAMEL: Communicative Agents for “Mind” Exploration of LLM Society

arxiv.org/abs/2303.17760 (2023-03)

问题：让两个 agent 协作完成任务时，对话很快退化（角色翻转、无限客套、提前宣布完成）。

核心方法：Inception Prompting——一个 task-specifier agent 先把模糊需求精炼成 specific task，然后 user-agent + assistant-agent 严格扮演角色循环对话。系统消息硬约束角色不可翻转，用结构化 Instruction:/Input：格式强制推进。

Takeaway：agent 间对话必须有强 protocol（固定字段 + 转换状态机），自由 chat 必崩。代码里给每条消息加 role+stage 字段、加防御性 prompt（“don't role-flip, don't say task done unless X”），并设硬性 turn cap——agent 间通信永远预设”对方会越界”。

4. ChatDev: Communicative Agents for Software Development

arxiv.org/abs/2307.07924 (2023-07)

问题：让 LLM 端到端写完整软件（不止 1 个文件），代码 hallucination 失控。

核心方法：瀑布式 pipeline（Design → Code → Test → Doc），每阶段双人对话（CTO+ 程序员、程序员 + 审查员）。引入 **communicative dehallucination**：审查 agent 主动提”含糊不清的细节”反问，强迫 coder 补全才能进下一步。

Takeaway: 长任务用 **chain-of-pairs** 而非单一 **super-agent**。每个交付物前插一个“adversarial reviewer”agent 追问细节——这比加 **reflection prompt** 有效得多，因为追问 agent 没有“快点完工”的偏置。代码上：把 **review pass** 设成 **pipeline** 的 **hard gate**。

5. Voyager: An Open-Ended Embodied Agent with Large Language Models

arxiv.org/abs/2305.16291 (2023-05)

问题: Agent 在开放环境 (Minecraft) 持续学习时，每个 session 重头开始，技能不沉淀。

核心方法: 三件套——**automatic curriculum** (GPT 自己提下一个目标) + **skill library** (把跑通的代码存成可用 function，向量检索复用) + **iterative prompting** (用环境反馈 + 错误信息自我修代码)。

Takeaway: 给你的 agent 加一个 **skill cache**——任何成功执行过的工具组合 (一段 Python、一个 SQL 查询、一个 shell 命令) 存成命名 function 写回 vector DB。下次类似任务先 **retrieve** 再生成。这就是把 agent 从“每次重新推理”变成“积累一个个人 utility 库”。

6. Generative Agents: Interactive Simulacra of Human Behavior

arxiv.org/abs/2304.03442 (2023-04)

问题: Long-running agent 没法维持一致的人格、关系、长期计划——上下文窗口装不下一周的记忆。

核心方法: 三层 memory 架构——**memory stream** (原始事件流) + **reflection** (定期对最近记忆做 abstraction，写成更高阶观察) + **planning** (基于 reflection 推导 daily/hourly plan)。检索分数 = $\text{recency} \times \text{importance} \times \text{relevance}$ 三者加权。

Takeaway: long-running agent 必须有分层 **memory**，不是简单的 vector store。代码层面：原始 log + LLM 周期性 summarize 成“insights”+ insights 再 summarize 成“identity/goals”。检索时把 **recency decay** 和 LLM 打的 importance score 加进相似度排序——不然永远召回最相似但最无关的旧事件。

7. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors

arxiv.org/abs/2308.10848 (2023-08)

问题: 固定 role 的 multi-agent 系统不灵活；任务不同需要的专家组合也不同。

核心方法: 4 阶段循环——**Expert Recruitment** (按任务动态 spawn 角色) → **Collaborative Decision** (垂直/水平讨论) → **Action Execution** → **Evaluation** (不达标退回重组)。Agent 阵容随任务自适应。

Takeaway: 不要硬编码 agent roster——先让一个“recruiter”agent 读任务，决定召哪些 **specialists**。代码里把 agent 定义抽成 registry (role_name → prompt + tools)，recruiter 输出 role 列表，runtime 实例化。这把“什么 agent 做什么”从配置文件升级到 LLM 决策，处理新任务零代码改动。

8. Mixture-of-Agents Enhances Large Language Model Capabilities

arxiv.org/abs/2406.04692 (2024-06)

问题: 单个 LLM 有 ceiling，但简单 ensemble (多次采样投票) 提升有限。

核心方法: 分层聚合——Layer 1 多个异构 LLM 并行回答 → Layer 2 LLM 把这些 response 当 reference 重新生成 (不是投票，是 综合改写) → 多层堆叠。开源 MoA 在 AlpacaEval 上超过 GPT-4o。

Takeaway: 质量瓶颈到了别堆 prompt——做 N-of-M 采样 + aggregator agent。代码模式：并行调用 3-5 个不同 model (Gemini Flash + Claude Haiku + GPT-5)，把所有 output 喂给一个”synthesizer”prompt 重新写。比 single-shot 大模型便宜且更稳，特别适合 critical 任务的最后一公里。

速查表

#	论文	一句话精髓
1	MetaGPT	Schema 化 artifact 通信 > 自由对话
2	AutoGen	Agent = (msg, sender) → reply 的纯函数
3	CAMEL	强 protocol 约束 + 硬性 turn cap
4	ChatDev	每个交付物前插 adversarial reviewer
5	Voyager	Skill cache: 成功的代码存进 vector DB
6	Generative Agents	三层 memory: raw → reflection → identity
7	AgentVerse	Recruiter agent 动态 spawn role
8	MoA	N-of-M 异构采样 + aggregator

推荐阅读顺序

新手：2 (AutoGen) → 1 (MetaGPT) → 4 (ChatDev) — 理解 multi-agent 基本范式
进阶：6 (Generative) → 5 (Voyager) — 理解记忆 + 学习
高阶：3 (CAMEL) → 7 (AgentVerse) → 8 (MoA) — 理解 protocol + 动态化 + 集成